Table Design – Sort Keys in Amazon Redshift

To Begin with the Lab

Summary of the Lab

In this lab, sort keys in **Amazon Redshift** were explored to understand how data ordering affects query performance. The automatically assigned sort key of an existing table was examined, and a new table was created using a custom compound sort key consisting of firstname and lastname columns. The assigned sort keys were then verified using Redshift system catalog views. Query execution plans were analyzed using the EXPLAIN command to understand how **Amazon Redshift** processes queries internally and estimates execution costs. Finally, the results were verified to understand how properly designed sort keys improve filtering, scanning efficiency, and overall query performance.

Understanding Sort Keys in Amazon Redshift

A sort key determines the physical order in which rows are stored on disk in **Amazon Redshift**. When queries filter, sort, or join using sort key columns, Redshift can scan fewer data blocks, reducing I/O and improving performance. Sort keys are especially useful for large analytical tables where queries frequently access data based on date ranges, customer information, or transaction details.

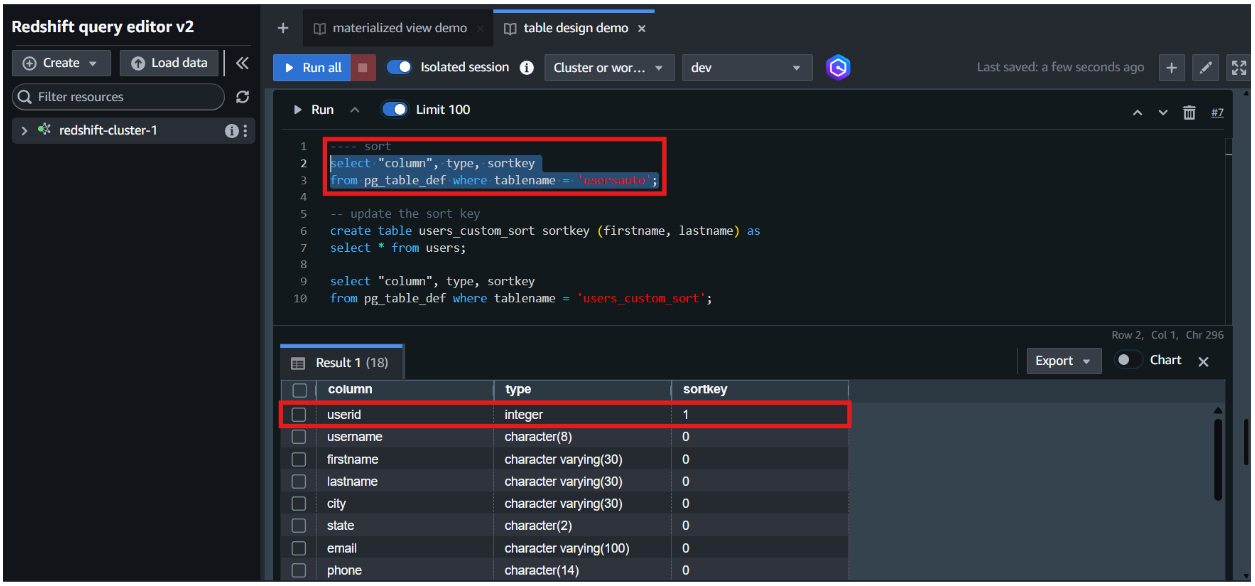
Example:

Consider an e-commerce Orders table containing millions of records sorted by order_date. When a query requests orders from the last 30 days, **Amazon Redshift** can quickly locate the required blocks instead of scanning the entire table. Without a sort key, every block must be scanned, resulting in slower query execution. Proper sort key selection therefore improves query efficiency and reduces processing time.

Lab Steps

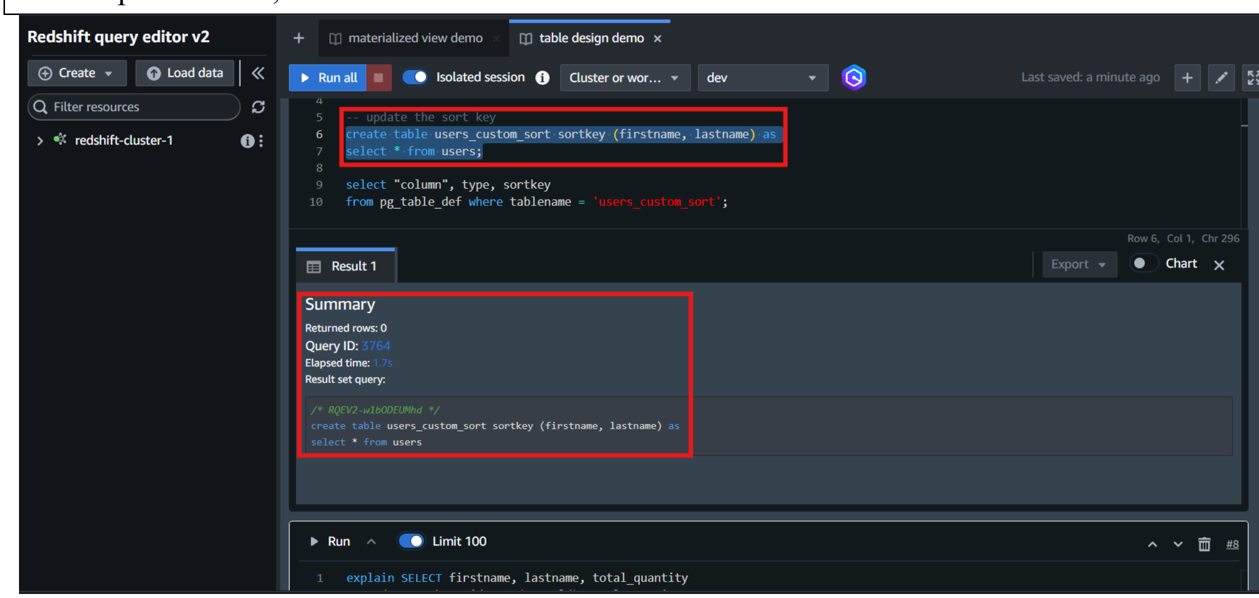
- Sign in to the AWS Management Console
- Navigate to **Amazon Redshift** → open **Query Editor v2**
- Review the existing sort key configuration of the usersauto table

```
SELECT tablename,
       "column",
       sortkey
FROM pg_table_def
WHERE tablename = 'usersauto';
```



- Verify that userid is automatically assigned as the sort key
- Understand that **Amazon Redshift** automatically selects sort keys when AUTO table optimization is enabled
- Create a new table with a compound sort key

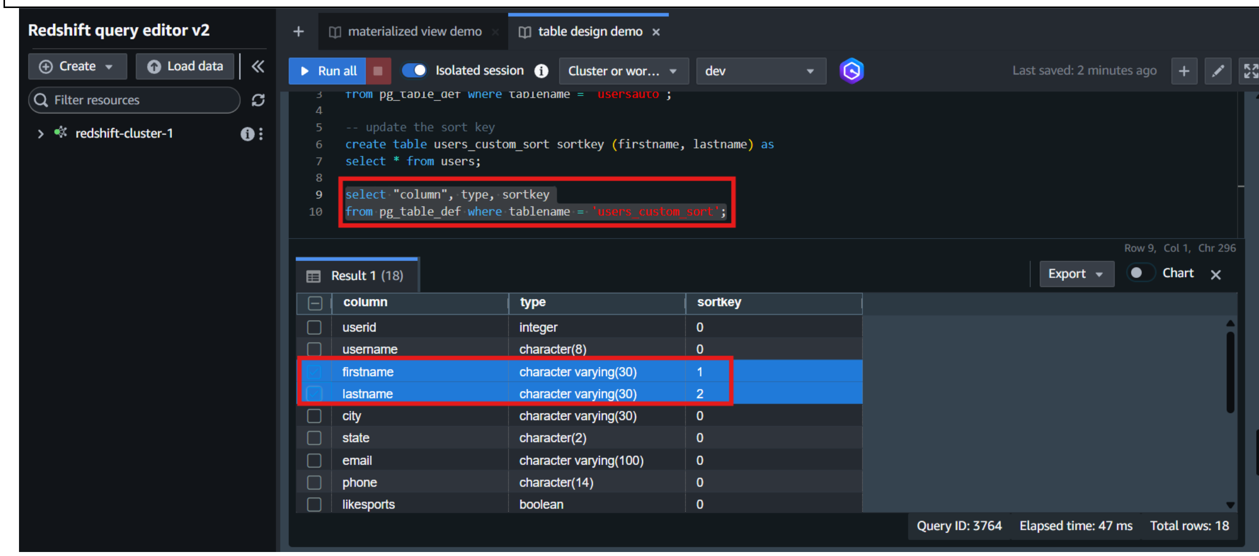
```
CREATE TABLE users_custom_sort
SORTKEY(firstname, lastname)
AS
SELECT *
FROM public.users;
```



- Click **Run**
- Verify that the table is created successfully
- Understand that firstname and lastname form a compound sort key
- Understand that compound sort keys prioritize columns from left to right during sorting

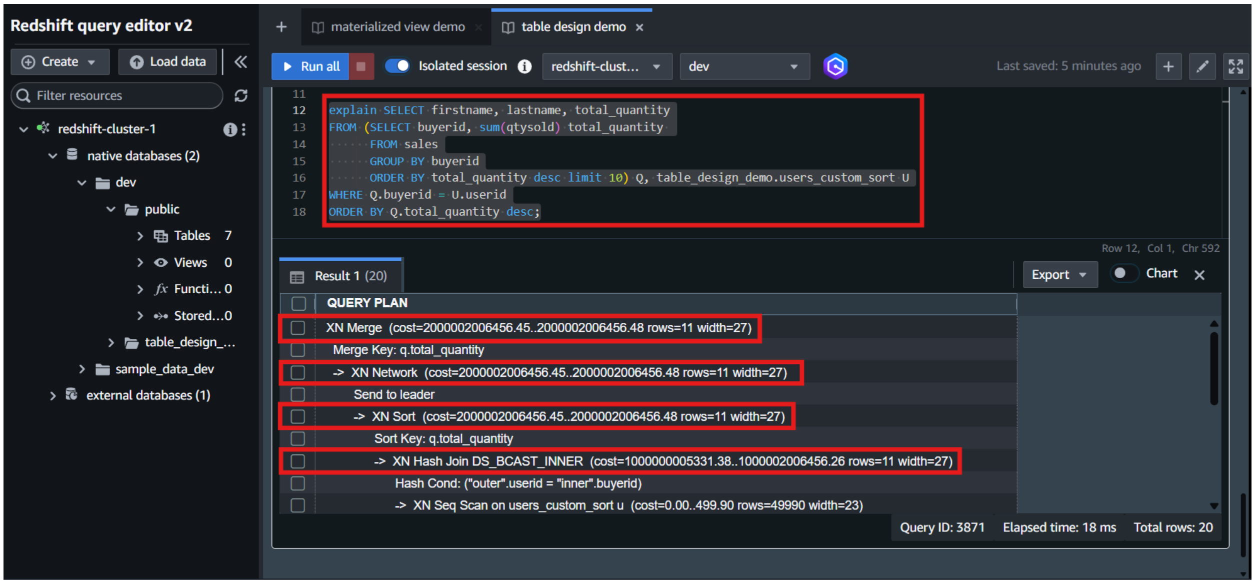
- Understand that INTERLEAVED SORTKEY can also be used when queries filter equally on multiple columns
- Verify the assigned sort keys

```
SELECT tablename,
       "column",
       sortkey
FROM pg_table_def
WHERE tablename = 'users_custom_sort';
```



- Verify that firstname is assigned sort key position 1
- Verify that lastname is assigned sort key position 2
- Analyze query performance using EXPLAIN

```
EXPLAIN
SELECT buyerid,
       SUM(qtysold)
FROM sales
GROUP BY buyerid
ORDER BY SUM(qtysold) DESC
LIMIT 10;
```



- Click **Run**
- Review the generated query execution plan
- Observe execution plan operations such as Merge, Network, and Sort
- Understand that EXPLAIN analyzes the execution plan without running the query
- Review important EXPLAIN metrics
    - Cost → Relative execution cost
    - Rows → Estimated rows returned
    - Width → Average row size
- Understand the Cost values returned by EXPLAIN
    - First value → Estimated cost to retrieve the first row
    - Second value → Estimated cost to return all rows
- Understand how sort keys improve query performance
    - Reduce data scanning
    - Improve filtering efficiency
    - Improve query execution performance
    - Reduce unnecessary disk reads
- Verify that firstname and lastname are successfully assigned as compound sort keys
- Verify that EXPLAIN generates a query execution plan successfully
- Verify that sort keys can be used to optimize query performance and reduce scan overhead in **Amazon Redshift**